# Da Semana 11 à Semana 12

### Conectando o HTML semântico, o localStorage, o fetch e as Browser APIs ao Node.js, ao npm, aos módulos e ao debugger

Desenvolvimento Front-End [React] T4 | Módulo 1 | Este material não traz conteúdo novo — ele mostra, ponto a ponto, como o que você já praticou nas semanas anteriores (módulos da S05, `async/await` da S06, DOM, e a S11 com `localStorage`, `fetch`, timers e revisão de código da IA) é a **base literal** do que vamos fazer na Semana 12. A ideia é simples: **nada aqui é assunto desconhecido, é o mesmo conhecimento crescendo.**

---

## Como ler este material

Cada seção segue o mesmo formato: primeiro, **o que você já sabe** (com o código como ele apareceu na Semana 11 e antes); depois, **o que a Semana 12 acrescenta**; por fim, uma comparação lado a lado entre a versão "de antes" e a "de agora" do mesmo problema.

> **O fio condutor: o programa que cresceu**
> Na Semana 11 você construiu o **PrevisaoCerta**, um painel que roda no navegador. Agora o palco muda: o JavaScript roda no **terminal**, com o **Node.js**, e o projeto é o **Quiz da Copa 2026**. O problema-âncora é de todo programador: *o programa cresceu — como instalar o que ele precisa, organizar em arquivos e descobrir o que está errado sem adivinhar?*

| Semana | O que você construiu | Ferramenta usada |
|---|---|---|
| 05 | Separar funções em arquivos no Node | `module.exports` / `require` |
| 06 | Código assíncrono sem travar a página | `async` / `await` |
| 08–09 | JavaScript mexendo na página; formulários | DOM, eventos, `submit` |
| 11 | Painel acessível, com memória e dados reais | HTML semântico, `localStorage` + JSON, `fetch`, `setTimeout` / `setInterval`, Geolocation |
| 12 | Quiz da Copa no terminal: pacotes, módulos e bugs caçados | **Assunto desta semana:** Node + npm, CommonJS × ESM, debugger do VS Code |

## CONEXÃO 1 — Dos quatro <script> em ordem aos módulos

### O que você já sabe (Semana 11)

No PrevisaoCerta você dividiu o código em `storage.js`, `api.js`, `ui.js` e `app.js` e os ligou no HTML **na ordem certa**. Tudo vivia no escopo global: se a ordem errasse, aparecia `... is not defined`.

```
<script src="js/storage.js"></script>
<script src="js/api.js"></script>
<script src="js/ui.js"></script>
<script src="js/app.js"></script>
```

### O que a Semana 12 acrescenta

- **Módulo:** cada arquivo diz o que **exporta** e o que **importa** — a dependência fica escrita no código, sem depender da ordem dos scripts.
- **Dois sistemas:** `module.exports` / `require` (CommonJS, o clássico do Node) e `export` / `import` (ESM, o padrão da linguagem, que também funciona no navegador).
- **Declare o sistema:** `"type": "module"` no `package.json` (ou `.mjs` / `.cjs`) — e, no ESM, a extensão `.js` é obrigatória.

### Comparando o código: o mesmo painel, antes e depois

**SEMANA 11 — ordem manual, tudo global**

```
// ui.js usa funções de storage.js
// "sem import": depende da ordem no HTML
function atualizarTela() {
  renderizarLista(carregarCidades());
}
```

**SEMANA 12 — dependências declaradas**

```
// ui.js (ESM)
import { carregarCidades } from "./storage.js";

export function atualizarTela() {
  renderizarLista(carregarCidades());
}
```

> **O que realmente mudou?**
>
> A lógica é a mesma. Muda **onde a dependência está escrita**: no HTML (frágil, pela ordem) ou no próprio arquivo (clara, verificável). No Módulo 2, o React vive de `import` / `export`.

## CONEXÃO 2 — Do localStorage e do JSON ao package.json e aos arquivos

### O que você já sabe (Semana 11)

`localStorage` guarda só texto; por isso usamos `JSON.stringify` para salvar e `JSON.parse` para ler. E na primeira visita a chave não existe: `getItem` devolve `null`, o que obriga o `|| []`.

### O que a Semana 12 acrescenta

- **O `package.json` é JSON:** o mesmo formato que você já lê e escreve. O npm o lê e o atualiza a cada `npm install`.
- **No Node não existe `localStorage`:** para guardar dados em disco usa-se o módulo nativo `node:fs` (`readFileSync` / `writeFileSync`) — com o mesmo par `stringify` / `parse`.
- **A "primeira visita" vira "o arquivo ainda não existe":** `existsSync` cumpre o papel do `|| []` (Ex4 de quinta).

### Comparando o código: a mesma lista, agora num arquivo

**SEMANA 11 — navegador**

```
function carregarCidades() {
  return JSON.parse(
    localStorage.getItem("previsaocerta:cidades")
  ) || [];
}
```

**SEMANA 12 — Node**

```
import { existsSync, readFileSync } from "node:fs";

export function carregarRanking() {
  if (!existsSync(ARQUIVO)) return [];
  return JSON.parse(readFileSync(ARQUIVO, "utf-
8"));
}
```

> **O que realmente mudou?**
>
> O raciocínio — texto ↔ estrutura, e o caso "ainda não existe" — é idêntico. Só muda **onde** o texto fica: na memória do navegador ou num arquivo do disco.

## CONEXÃO 3 — Do console, dos erros e do DevTools ao debugger do VS Code

### O que você já sabe (Semanas 08–11)

Você já lê o **Console**, usa a aba **Application** para conferir o `localStorage`, a aba **Network** para ver o `fetch` e já aprendeu a ler mensagens como `Cannot read properties of null`. Isso é depuração — só que feita "no olho" e com `console.log`.

### O que a Semana 12 acrescenta

- **Novas mensagens para ler no terminal:** `Cannot find module` (faltou `npm install`), `ERR_MODULE_NOT_FOUND` (faltou o `.js`), `require is not defined` (projeto ESM) e o `TypeError ... (reading 'yellow')` do chalk.
- **Breakpoint no lugar do `console.log`:** pausar o programa e olhar **Variables**, **Watch** e **Call Stack**, sem editar o código.
- **A pista que o `console.log` esconde:** `console.log("3", 3)` imprime `3 3`; o painel Variables mostra `"3"` com aspas — é texto.

| No navegador (Semana 11) | No Node + VS Code (Semana 12) |
|---|---|
| `console.log` e a aba Console | Breakpoint, Watch e Debug Console |
| Aba Sources e `debugger;` | Run and Debug, `launch.json` e o JavaScript Debug Terminal |
| Aba Application (chaves do `localStorage`) | `package.json`, `package-lock.json` e `node_modules` |
| Aba Network (status 200/404) | `npm ls`, `npm outdated` e `npm audit` |
| Ler "Cannot read properties of null" | Ler "Cannot read properties of undefined" e achar o `q` indefinido pela Call Stack |

## CONEXÃO 4 — Das Browser APIs ao ambiente do Node

### O que você já sabe (Semana 11)

`setTimeout`, `setInterval`, `fetch`, `localStorage` e Geolocation são **APIs do navegador**. Você as usou com `window` e `document` sempre por perto.

### O que a Semana 12 acrescenta

O JavaScript é o mesmo, mas **o que está disponível depende do palco**:

| Recurso | Navegador | Node.js |
|---|---|---|
| `console.log`, `setTimeout`, `setInterval` | sim | sim |
| `fetch` | sim | sim (versões recentes) |
| `window`, `document`, DOM, `localStorage`, Geolocation | sim | **não** |
| `process`, `require`, `node:fs`, `node:path` | **não** | sim |
| `import` / `export` (ESM) | sim ( `type="module"` ) | sim ( `"type":"module"` ) |

```
console.log(typeof window, typeof process);
// navegador → object undefined
// Node      → undefined object
```

## CONEXÃO 5 — Da revisão do código da IA à revisão de pacotes e módulos

### O que você já sabe (Semana 11)

O ciclo **gerar → revisar → consertar → justificar** e os pontos cegos da IA: acessibilidade, segurança, tratamento de erro e API depreciada ou alucinada. A regra: *você não usa um código que não consegue explicar*.

### O que a Semana 12 acrescenta

- **Pacotes que não existem:** a IA pode sugerir um nome inventado — confira no npmjs.com (nome, downloads, mantenedor) antes de instalar.
- **Mistura de sistemas:** `import` e `require` no mesmo arquivo; `.js` esquecido no `import`; `__dirname` no ESM.
- **Instalação global por hábito:** a IA sugere `-g` ou `sudo` onde o certo é local.
- **O debugger como prova:** para explicar e justificar um conserto, mostre o breakpoint e as variáveis.

> **Ligação direta**
> A atividade da quinta usa exatamente isto: a IA entrega um `index.js` com `import` e `require` misturados — e você diagnostica a partir da mensagem de erro.

## Checklist: você já está pronto para a Semana 12 se...

- ✓ Sabe abrir o terminal do VS Code, trocar de pasta (`cd`) e listar arquivos.
- ✓ Sabe escrever funções, laços (`for`), `if/else`, template strings e arrays de objetos.
- ✓ Lembra de `module.exports` e `require` da Semana 05 (vamos retomar na quinta).
- ✓ Entende que o `localStorage` só guarda texto e usa `JSON.stringify` / `JSON.parse`.
- ✓ Sabe o que faz `async` / `await` e `try/catch`.
- ✓ Sabe ler uma mensagem de erro inteira e achar o arquivo e a linha.
- ✓ Sabe abrir o DevTools (F12) e olhar Console e Sources.

Se todos esses pontos fazem sentido, a Semana 12 não vai apresentar nada desconhecido — ela dá nome, comando e ferramenta profissional (npm, módulos, debugger) para instintos que você já desenvolveu.